

FAILSAFE Project Report

Abstract

Schools and colleges often face a big challenge: they only find out a student is failing after the final exams are over. By then, it is too late to help them. This report explains my project, **FAILSAFE**. It is a Machine Learning system designed to predict if a student is at risk of failing before the final exams. More importantly, it does not just say "Yes" or "No". It tells teachers exactly *why* a student is struggling (for example, low study time or too many absences) and suggests what actions the school should take. This report will explain the steps I took to build this system in a simple, easy-to-understand way.

1 Understanding and Preparing the Data

Machine learning models learn from data. For this project, I used a dataset that contained records of 395 students. For each student, the dataset had 33 different pieces of information, such as their parents' education, internet access at home, study time, absences, and previous failures.

1.1 Preventing Cheating (Data Leakage)

The dataset originally contained grades from the first term and second term. I had to remove these grades before training the model. Because if the model knows a student failed the first two terms, it becomes too easy to guess they will fail the final term. The model wouldn't learn anything about their actual habits. By removing these grades, I forced the model to learn the real behavioral reasons behind failing.

1.2 Converting Text to Numbers

Machine learning algorithms are essentially math formulas—they only understand numbers. However, my dataset had text categories, like asking if a student had extra school support.

I used a simple technique called **One-Hot Encoding**. This technique changes text categories into 1s and 0s. For example, a yes becomes a 1, and a no becomes a 0. After

making all these conversions, my dataset expanded from 33 columns to 56 columns of pure numbers, perfectly ready for the machine learning model.

2 The Machine Learning Model

The brain of the FAILSAFE system is an algorithm called **XGBoost** (Extreme Gradient Boosting).

2.1 Why XGBoost?

Imagine you are trying to guess how many jellybeans are in a jar. If you ask one person, they might be very wrong. But if you ask 100 people and take the average of their guesses, you will be much closer to the right answer. XGBoost works similarly. Instead of using one big decision tree to make a prediction, it creates many small, weak decision trees. Each new tree tries to fix the mistakes made by the previous trees. Eventually, they all work together as a team to make a very accurate prediction.

2.2 Handling the Imbalance

There was a small problem in my data: only about 33% of the students were actually failing. The other 67% were passing safely. Because there were so many more passing students, the model might just lazily guess Safe for everyone and still get a high accuracy score.

To fix this, I changed the model's rules. I gave a much higher penalty to the model if it made a mistake on a failing student. Basically, I told the model: It is okay to accidentally flag a safe student, but it is absolutely unacceptable to miss a student who is actually failing.

3 Did the Model Work? (Results)

To test if the model actually worked, I hid 20% of the data from it. I trained the model on the 80%, and then gave it a test on the hidden 20%.

In machine learning, we use a score called **ROC-AUC** to see how well the model separates the two groups. A score of 0.5 means the model is just guessing randomly. A score of 1.0 means the model is perfect. My model achieved a ROC-AUC score of 0.70. This means the model did a good job of learning the difference between passing and failing students.

When looking at the exact numbers, out of the students it tested, it correctly found 15 students who were going to fail. It did miss 11 students, but finding those 15 students

means 15 people can now get the help they need before it is too late.

4 Making the AI Explainable (SHAP)

This is the most important part of the project. A big problem with powerful AI models is that they are "black boxes." They give you an answer, but they don't tell you *how* they got that answer. If the system tells a principal, Student A is going to fail, the principal will ask, "Why?" If the AI can't answer, it is useless.

To fix this, I used a tool called SHAP. SHAP is a clever way to peek inside the black box. It looks at the prediction and breaks it down. It can tell us: "Student A is predicted to fail, and the top reason is because they have 15 absences, followed by a history of past class failures."

By using SHAP, I turned a mysterious AI prediction into a clear, understandable report that humans can trust.

5 Taking Action: The FastAPI Backend

Predicting risk is only half the battle; the other half is doing something about it. I built a backend system using **FastAPI** to automatically turn the model's predictions into real-world action plans.

Here is how the system works in plain English:

1. The school uploads a file with student data.
2. The XGBoost model calculates the risk percentage for every student.
3. The SHAP tool figures out the exact reasons why each student is at risk.
4. The system then looks at my custom "Intervention Rules".
 - If SHAP says the main problem is past **failures**, the system automatically suggests scheduling a "**Counseling session**".
 - If the main problem is **absences**, the system suggests "**Attendance monitoring**".
 - If the main problem is low **study time**, it suggests making a "**Structured study schedule**".
5. Finally, the system creates a neat, easy-to-read Excel/CSV file and gives it back to the school administrators.

6 Conclusion

The FAILSAFE project shows that machine learning doesn't have to be a scary, confusing black box. By combining XGBoost for smart predictions, SHAP for clear explanations, and a FastAPI system for automated rules, we can create a tool that is actually useful for schools.

This system bridges the gap between complex computer code and real human support, ensuring that students get the exact help they need, right when they need it.